IntelliLight v2.0 - Product Requirements Document (PRD)

Multi-Agent ILPS Architecture + Real-Time Admin Control System

Version: 2.0.0
Date: March 11, 2026
Status: Pre-Development Planning
Target: Smart India Hackathon 2025 Demo + Production Deployment

---

Executive Summary

This PRD outlines the evolution of IntelliLight from a centralized 3-junction corridor controller to a **scalable city-wide multi-agent system** with **real-time human oversight capabilities**. The upgrade consists of two major phases:

Phase 1 (Weeks 2-3): Migrate to Independent Learning with Parameter Sharing (ILPS)
Phase 2 (Week 4): Build Admin Panel with real-time control and green-wave override

The system will maintain the current 58% performance advantage while enabling:

- ✅ Zero-retraining scalability to 100+ intersections
- ✅ Real-time human override for emergency situations
- ✅ Manual green-wave corridor creation for convoys/events
- ✅ Per-phase decision making (replacing 4-phase batch decisions)

Table of Contents

1. Current System Analysis
2. Problems & Limitations
3. Proposed Solution Overview
4. Phase 1: ILPS Multi-Agent Migration
5. Phase 2: Admin Control Panel
6. Technical Architecture
7. Implementation Timeline
8. Success Metrics
9. Risk Assessment
10. Appendix: Technical Specifications

---

1. Current System Analysis

1.1 Architecture Overview

Current Model: Centralized PPO Controller
Observation Space: `Box(42,)` — 3 junctions × 14 features
Action Space: `MultiDiscrete([8] × 12)` — 3 junctions × 4 phases
Decision Frequency: Once per full cycle (all 4 phases planned upfront)

Current Workflow:

```
1. Agent observes ALL 3 junctions simultaneously [42 features]
2. Agent decides green durations for ALL 12 phases (3 junctions × 4 phases)
3. Execute Phase 0 (EW-Through) at all 3 junctions
4. Execute Phase 1 (EW-Left) at all 3 junctions
5. Execute Phase 2 (NS-Through) at all 3 junctions
6. Execute Phase 3 (NS-Left) at all 3 junctions
   [30-60 seconds elapsed before next decision]
7. Repeat from step 1
```

1.2 Current Performance

Metric	vs Max-Pressure	vs Fixed-Timer
Wait Time	**-58%**	**-78%**
Queue Length	**-53%**	**-78%**
Starvation Events	**0**	**0**

1.3 Current Limitations

L1: Batch Decision Rigidity

- Agent commits to 4-phase durations upfront
- Cannot adapt if traffic changes mid-cycle
- Example: Phase 0 starts with 5 cars, but 20 arrive during Phase 1 → still executes original Phase 2 duration

L2: Fixed Network Size

- Hardcoded to 3 junctions (42 dims)
- Adding/removing intersections requires full retraining
- Cannot deploy on different city layouts without model changes

L3: No Human Override

- No manual control during emergencies
- Cannot create green-wave corridors for convoys
- No way to adjust model decisions in real-time

L4: Minimum Green Time = 10s

- Too short for pedestrian safety in real deployments
- Industry standard is 15-25s depending on crosswalk length

---

2. Problems & Limitations

2.1 Problem 1: Delayed Response to Traffic Changes

Current Behavior:

```
t=0s:   Agent sees [Q_EW=5, Q_NS=3] → Decides [30s, 15s, 25s, 20s]
t=10s:  (Executing Phase 0: EW-Through 30s)
t=15s:  20 new vehicles arrive in NS queue
t=30s:  (Executing Phase 1: EW-Left 15s) ← Still using old decision!
t=45s:  (Executing Phase 2: NS-Through 25s) ← Finally serves NS, but already
delayed
```

Impact:

- Suboptimal response to traffic spikes
- Missed opportunities to shorten/extend phases dynamically
- Performance degradation during volatile traffic

Proposed Solution: Per-phase decision making (see Section 4.3)

2.2 Problem 2: Cannot Scale to City-Wide Deployment

Current Constraint:

- Observation: `Box(42,)` hardcoded for 3 junctions
- Action: `MultiDiscrete([8] × 12)` hardcoded for 3 junctions × 4 phases
- Adding Junction 4 requires:
  - Redefine observation to `Box(56,)`
  - Redefine action to `MultiDiscrete([8] × 16)`
  - **Retrain entire model from scratch** (200K steps × 2-3 hours)

Deployment Blocker:

- Each city layout needs custom training
- Cannot deploy pre-trained model to new intersections
- Maintenance nightmare (retrain every time road closes/opens)

Proposed Solution: ILPS architecture where every junction runs the same 14→4 brain (see Section 4.2)

2.3 Problem 3: No Human Control for Special Events

Real-World Scenarios:

Scenario A: Presidential Convoy

- Convoy route: J1 → J2 → J3 (entire corridor)
- Requirement: All signals green in convoy direction
- Current system: Cannot override RL decisions

Scenario B: Emergency Response

- Fire truck needs J2→J3 corridor
- Model decides J2 should stay red for 30s
- No way to manually force green

Scenario C: Model Gives Bad Decision

- RL decides 45s green for empty lane
- Human operator sees mistake on camera
- Cannot correct in real-time

Proposed Solution: Admin panel with real-time override (see Section 5)

2.4 Problem 4: Minimum Green = 10s Too Short

Current Setting: `signal.MIN_GREEN = 10`

Industry Standards:

- **Pedestrian Safety:** 15-25s (depends on crosswalk width)
- **Vehicle Clearance:** 12-15s minimum
- **Real Deployments:** Typically 20-25s

Risk:

- 10s insufficient for pedestrians to cross safely
- Regulatory rejection in real deployment

Proposed Solution: Change to `MIN_GREEN = 20` (see Section 4.6)

---

3. Proposed Solution Overview

3.1 Vision

Transform IntelliLight into a **production-grade city-wide adaptive traffic control platform** with:

1. **Zero-retraining scalability** — Add/remove intersections without model changes
2. **Real-time adaptability** — Make decisions every phase, not every cycle
3. **Human-in-the-loop control** — Override RL for emergencies and special events
4. **Industry-standard safety** — 20s minimum green for regulatory compliance

3.2 Two-Phase Development Plan

Phase 1: ILPS Multi-Agent Migration (Weeks 2-3)

- Refactor environment to per-junction observations
- Train universal junction brain (14 dims → 4 actions)
- Enable parameter sharing across all intersections
- Implement per-phase decision making
- Increase MIN_GREEN to 20s

Phase 2: Admin Control Panel (Week 4)

- Build web-based real-time dashboard
- Implement manual override system (±5s adjustment window)
- Create green-wave corridor builder
- Add city map visualization
- Integrate with RL backend via WebSocket

3.3 Success Criteria

Phase 1 Complete When:

- ✅ Model trained on 3 junctions can control 9 without retraining
- ✅ Per-phase decisions improve response time by 20%+
- ✅ Performance maintained (wait time improvement ≥ 50%)
- ✅ All intersections respect 20s minimum green

Phase 2 Complete When:

- ✅ Admin can override any phase decision within 5s window
- ✅ Green-wave corridors can be created via GUI
- ✅ Dashboard shows live traffic state and upcoming decisions
- ✅ System handles admin override without crashes

---

4. Phase 1: ILPS Multi-Agent Migration

4.1 ILPS Architecture Fundamentals

Core Concept: Instead of one centralized brain controlling all junctions, train a **single universal junction brain** that controls ONE intersection. Deploy copies of this brain to every intersection in the city.

Current (Centralized):

```
Global Obs [42] → PPO Brain → Global Actions [12]
(3 junctions × 14 features) → Network → (3 junctions × 4 phases)
```

Proposed (ILPS):

```
J1 Local Obs [14] → Shared Brain → J1 Actions [4]
J2 Local Obs [14] → Shared Brain → J2 Actions [4]
J3 Local Obs [14] → Shared Brain → J3 Actions [4]
...
JN Local Obs [14] → Shared Brain → JN Actions [4]
```

Key Insight: The brain doesn't need to know if it's controlling J1 or J42 — it just needs to know:

- How many cars are waiting in each direction
- What phase is currently active
- Whether emergency vehicle is present

4.2 Observation Space Enhancement

Current Local Observation (14 dims):

```
[queues_N, queues_S, queues_E, queues_W,          # 4
 delta_N, delta_S, delta_E, delta_W,              # 4
 wait_N, wait_S, wait_E, wait_W,                  # 4
 emergency_flag,                                   # 1
 current_phase]                                    # 1
```

Enhanced Local Observation (18 dims):

```
[queues_N, queues_S, queues_E, queues_W,          # 4
 delta_N, delta_S, delta_E, delta_W,              # 4
 wait_N, wait_S, wait_E, wait_W,                  # 4
 emergency_flag,                                   # 1
 current_phase,                                    # 1
 upstream_phase,                                   # 1  ← NEW: What phase is
J(i-1) in?
 downstream_phase,                                 # 1  ← NEW: What phase is
J(i+1) in?
 time_since_upstream_switch,                       # 1  ← NEW: Offset for green
wave
 time_since_downstream_switch]                     # 1  ← NEW: Coordination signal
```

Why Add Neighbor Features?

- Enables green-wave coordination without centralized control
- Agent learns: "If upstream is in Phase 0, I should enter Phase 0 in 10s"
- Natural emergence of progressive signal timing

For Edge Junctions:

- If no upstream: set `upstream_phase = -1, time_since_upstream_switch = -1`
- If no downstream: set `downstream_phase = -1, time_since_downstream_switch = -1`

4.3 Per-Phase Decision Making

Current (Batch Decisions):

```python
# Agent decides all 4 phases at once
action = [2, 1, 3, 2]  # Indices: [20s, 15s, 25s, 20s]

# Execute entire cycle
for phase_idx in range(4):
    duration = GREEN_DURATIONS[action[phase_idx]]
    execute_phase(phase_idx, duration)  # ← Cannot change mid-cycle
```

Proposed (Per-Phase Decisions):

```python
# Make decision only for NEXT phase
while not done:
    obs = get_current_observation()
    phase_to_execute = get_next_phase()  # Cyclic: 0→1→2→3→0

    action = agent.predict(obs)  # Decide duration for THIS phase only
    duration = GREEN_DURATIONS[action]

    execute_phase(phase_to_execute, duration)
    # ← Agent makes NEW decision after each phase completes
```

Benefits:

Benefit 1: Real-Time Adaptation

```
t=0s:   Agent sees Q_NS=3 → Decides Phase 2 (NS) = 20s
t=20s:  (Phase 2 executing...)
t=25s:  15 new vehicles arrive in NS queue
t=40s:  Agent sees Q_NS=18 → Decides Phase 3 (NS-left) = 35s ← Adapts!
```

Benefit 2: Emergency Response

```
t=0s:   Phase 0 executing (EW-Through 30s)
t=15s:  Emergency vehicle detected in NS direction
t=30s:  Agent sees emergency_flag=1 → Decides Phase 2 (NS) = 45s + bonus
        ← Responds within one phase (10-45s) instead of full cycle (60-120s)
```

Benefit 3: Smoother Learning

- Single decision per step → simpler action space
- Faster reward feedback (10-45s vs 60-120s)
- Easier to debug bad decisions

Implementation:

```python
class PerPhaseCorridorEnv(gym.Env):
    def __init__(self, n_junctions=3):
        self.n_junctions = n_junctions

        # CHANGE: Action space is now [8] instead of [8]×12
        # Agent controls ONE phase at ONE junction per step
        self.action_space = spaces.Discrete(8)  # Select duration index

        # Observation includes which junction is active
        self.observation_space = spaces.Box(
            low=0, high=1,
            shape=(18 + 1,),  # Local obs (18) + junction_id (1)
            dtype=np.float32
        )

        self.current_junction_idx = 0
        self.current_phase_idx = 0

    def step(self, action):
        duration_idx = action
        duration = GREEN_DURATIONS[duration_idx]

        # Execute phase at current junction
        execute_phase(
            junction=self.current_junction_idx,
            phase=self.current_phase_idx,
            duration=duration
        )

        # Move to next junction's same phase
        self.current_junction_idx += 1

        # If we completed all junctions for this phase, move to next phase
        if self.current_junction_idx >= self.n_junctions:
            self.current_junction_idx = 0
            self.current_phase_idx = (self.current_phase_idx + 1) % 4

        # Get observation for next junction
        obs = self._get_observation()
        reward = self._calculate_reward()

        return obs, reward, done, truncated, info
```

Execution Pattern:

```
Step 1: Decide Phase 0 duration for J1 → Execute
Step 2: Decide Phase 0 duration for J2 → Execute
Step 3: Decide Phase 0 duration for J3 → Execute
Step 4: Decide Phase 1 duration for J1 → Execute
Step 5: Decide Phase 1 duration for J2 → Execute
...
Step 12: Decide Phase 3 duration for J3 → Execute
Step 13: Decide Phase 0 duration for J1 → Execute (cycle repeats)
```

Question: Does this slow down decision-making?

Answer: No, actually FASTER response time:

Current approach:

- Decision every ~90s (full cycle)
- Reaction time: 0-90s depending on when event occurs in cycle

Per-phase approach:

- Decision every ~20-35s (one phase)
- Reaction time: 0-35s maximum
- **50-60% faster response to changes**

4.4 Parameter Sharing Implementation

Training Setup:

```python
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv

# Create training environment with 3 junctions
env = DummyVecEnv([lambda: PerPhaseCorridorEnv(n_junctions=3)])

# Train PPO on this environment
model = PPO(
    "MlpPolicy",
    env,
    learning_rate=3e-4,
    n_steps=2048,
    batch_size=64,
    # ... other hyperparameters same as before
)

model.learn(total_timesteps=200000)
model.save("models/ilps_universal_junction_brain.zip")
```

Deployment to 9-Junction Grid:

```python
# ZERO retraining needed!
model = PPO.load("models/ilps_universal_junction_brain.zip")

# Create 9-junction environment
env_9 = PerPhaseCorridorEnv(n_junctions=9)

# Model works immediately because:
# - Observation shape is still (19,) — same as training
# - Action space is still Discrete(8) — same as training
# - Agent doesn't care that there are 9 junctions instead of 3

obs = env_9.reset()
for _ in range(1000):
    action, _states = model.predict(obs)
    obs, reward, done, truncated, info = env_9.step(action)
```

Key Advantage:

Task	Centralized Approach	ILPS Approach
Train on 3 junctions	200K steps	200K steps
Deploy to 9 junctions	**Retrain 200K steps**	**0 steps** ✓
Deploy to 25 junctions	**Retrain 200K steps**	**0 steps** ✓
Deploy to different city	**Retrain 200K steps**	**0 steps** ✓

4.5 Curriculum Learning for ILPS

Stage 1: Single Junction Mastery (0-66K steps)

```python
env = PerPhaseCorridorEnv(n_junctions=1)
scenario = "WEEKEND"  # Light traffic
```

- Learn basic 4-phase control
- Master per-phase decision making
- Understand observation→action mapping

Stage 2: Corridor Coordination (66K-133K steps)

```python
env = PerPhaseCorridorEnv(n_junctions=3)
scenario = "EVENING_RUSH"  # Medium traffic
```

- Learn to use neighbor phase features
- Develop green-wave coordination
- Handle medium traffic density

Stage 3: Full Complexity (133K-200K steps)

```python
env = PerPhaseCorridorEnv(n_junctions=3)
scenario = "MORNING_RUSH"  # Heavy traffic + incidents
```

- Master heavy traffic scenarios
- Handle emergency vehicles
- Deal with random incidents

4.6 Minimum Green Time Change

Implementation:

```python
# In configs/parameters.py

class SignalConfig:
    # OLD
    # MIN_GREEN = 10

    # NEW
    MIN_GREEN = 20  # Industry standard for pedestrian safety

    # Green duration options adjusted accordingly
    GREEN_DURATIONS = [20, 25, 30, 35, 40, 45, 50, 55]  # All ≥ 20s
```

Action Space Mapping:

```
Old: [10, 15, 20, 25, 30, 35, 40, 45]
New: [20, 25, 30, 35, 40, 45, 50, 55]
```

Impact on Reward:

- Slightly longer minimum phase → less flexibility
- But: necessary for real-world safety compliance
- Expected performance: Still 50%+ improvement vs baselines

4.7 Expected Performance After ILPS Migration

Hypothesis:

Metric	Current (Centralized)	ILPS (Per-Phase)	Change
Wait Time vs Max-Pressure	-58%	**-60% to -65%**	+2-7% improvement
Queue Length	-53%	**-55% to -60%**	+2-7% improvement
Response Time	45-90s	**20-35s**	50% faster
Scalability	3 junctions only	**Unlimited**	∞

Why Expect Better Performance?

1. **Faster adaptation** — Decisions every 20-35s instead of 90s
2. **More training data** — 3× junctions = 3× experience per episode
3. **Better credit assignment** — Reward immediately after each phase
4. **Emergency response** — Can prioritize within 20s instead of 90s

4.8 ILPS Migration Checklist

Week 2 Tasks:

- [ ] **Day 1-2: Environment Refactoring**
- [ ] Create `PerPhaseCorridorEnv` class
- [ ] Implement 18-dim enhanced observation
- [ ] Add neighbor phase tracking
- [ ] Update reward calculation for per-phase
- [ ] Write unit tests
- [ ] **Day 3-4: Training Pipeline**
- [ ] Update training script for ILPS
- [ ] Implement 3-stage curriculum
- [ ] Add per-phase logging
- [ ] Configure TensorBoard metrics
- [ ] **Day 5-7: Training & Validation**
- [ ] Train universal brain (200K steps)
- [ ] Validate on 3-junction corridor
- [ ] Test on 9-junction grid (zero retraining)
- [ ] Compare performance vs centralized

Week 3 Tasks:

- [ ] **Day 1-2: Optimization**
- [ ] Hyperparameter tuning if needed
- [ ] Performance profiling
- [ ] Memory optimization for large grids
- [ ] **Day 3-5: Evaluation**
- [ ] Run full evaluation suite (20 eps × 3 scenarios)
- [ ] Stress test with 25-junction grid
- [ ] Document performance improvements
- [ ] **Day 6-7: Documentation**
- [ ] Update README with ILPS architecture
- [ ] Write ILPS technical guide
- [ ] Prepare demo for Phase 2 kickoff

---

5. Phase 2: Admin Control Panel

5.1 System Architecture

Components:

```
┌──────────────────────────────────────────────────────────┐
│                    Admin Control Panel                   │
│                     (React Frontend)                     │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐   │
│  │  City Map    │  │ Live Traffic │  │ Green Wave   │   │
│  │  Visualizer  │  │  Dashboard   │  │   Builder    │   │
│  └──────────────┘  └──────────────┘  └──────────────┘   │
└──────────────────────────────────────────────────────────┘
                        │ WebSocket (Real-time)
                        │
┌──────────────────────────────────────────────────────────┐
│                 Control Server (Python)                  │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐   │
│  │  Override    │  │  Green Wave  │  │   Decision   │   │
│  │  Manager     │  │  Controller  │  │   Monitor    │   │
│  └──────────────┘  └──────────────┘  └──────────────┘   │
└──────────────────────────────────────────────────────────┘
                        │ Python API
                        │
┌──────────────────────────────────────────────────────────┐
│                 RL Environment (SUMO)                    │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐   │
│  │   Junction   │  │   Junction   │  │   Junction   │   │
│  │      J1      │  │      J2      │  │      J3      │   │
│  └──────────────┘  └──────────────┘  └──────────────┘   │
└──────────────────────────────────────────────────────────┘
```

5.2 Feature 1: Real-Time Decision Override

5.2.1 User Story

As a traffic control operator
I want to manually adjust RL-decided phase durations
So that I can correct obvious mistakes or handle unexpected situations

Acceptance Criteria:

- Admin sees upcoming phase decision 5 seconds before execution
- Admin can increase/decrease duration by ±10 seconds
- Override window closes after 5 seconds
- System logs all manual overrides for audit

5.2.2 Workflow

```
RL Agent Decision:
t=0s:  Agent predicts Phase 2 (NS-Through) → 40 seconds
       → Send to Admin Panel via WebSocket

Admin Panel Display:
t=0s:  [Alert] Upcoming Phase: NS-Through (40s) at J2
       [Slider] Adjust: 30 35 40 45 50 seconds
       [Timer] Override window: 5... 4... 3... 2... 1...

Admin Actions (if needed):
t=2s:  Operator sees heavy NS queue on camera
       → Moves slider to 50 seconds
       → Clicks "Apply Override"
       → Override sent to Control Server

Execution:
t=5s:  Override window closes
       → If override sent: Execute Phase 2 with 50s
       → If no override: Execute Phase 2 with original 40s (RL decision)
```

5.2.3 Technical Implementation

Backend (Python FastAPI):

python

```python
from fastapi import FastAPI, WebSocket
from datetime import datetime
import asyncio

app = FastAPI()

class OverrideManager:
    def __init__(self):
        self.pending_decisions = {}  # {junction_id: (phase, duration, timestamp)}
        self.overrides = {}  # {junction_id: new_duration}
        self.override_window = 5  # seconds

    async def register_decision(self, junction_id, phase, duration):
        """Called when RL agent makes a decision"""
        timestamp = datetime.now()
        self.pending_decisions[junction_id] = (phase, duration, timestamp)

        # Broadcast to all connected admin panels
        await self.broadcast_decision(junction_id, phase, duration)

        # Wait for override window
        await asyncio.sleep(self.override_window)

        # Check if override was submitted
        if junction_id in self.overrides:
            final_duration = self.overrides[junction_id]
            del self.overrides[junction_id]
            self.log_override(junction_id, phase, duration, final_duration)
            return final_duration
        else:
            return duration  # Use original RL decision

    def submit_override(self, junction_id, new_duration):
        """Called when admin submits override"""
        if junction_id in self.pending_decisions:
            phase, old_duration, timestamp = self.pending_decisions[junction_id]

            # Check if still within override window
            elapsed = (datetime.now() - timestamp).total_seconds()
            if elapsed < self.override_window:
                self.overrides[junction_id] = new_duration
                return {"status": "accepted", "new_duration": new_duration}
            else:
                return {"status": "rejected", "reason": "override_window_expired"}
        else:
```

```
        else:
            return {"status": "rejected", "reason": "no_pending_decision"}

override_manager = OverrideManager()

@app.websocket("/ws/admin")
async def admin_websocket(websocket: WebSocket):
    await websocket.accept()
    # Register websocket for broadcasts
    # ... implementation

@app.post("/override/{junction_id}")
async def submit_override(junction_id: str, new_duration: int):
    result = override_manager.submit_override(junction_id, new_duration)
    return result
```

Frontend (React):

javascript

```javascript
// AdminPanel.jsx
import React, { useState, useEffect } from 'react';

function DecisionOverride() {
    const [pendingDecision, setPendingDecision] = useState(null);
    const [timeRemaining, setTimeRemaining] = useState(0);
    const [newDuration, setNewDuration] = useState(null);

    useEffect(() => {
        const ws = new WebSocket('ws://localhost:8000/ws/admin');

        ws.onmessage = (event) => {
            const data = JSON.parse(event.data);
            if (data.type === 'decision') {
                setPendingDecision(data);
                setNewDuration(data.duration);
                setTimeRemaining(5);

                // Countdown timer
                const interval = setInterval(() => {
                    setTimeRemaining(prev => {
                        if (prev <= 1) {
                            clearInterval(interval);
                            setPendingDecision(null);
                            return 0;
                        }
                        return prev - 1;
                    });
                }, 1000);
            }
        };

        return () => ws.close();
    }, []);

    const handleOverride = async () => {
        const response = await fetch(
            `/override/${pendingDecision.junction_id}`,
            {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ new_duration: newDuration })
            }
        );
        const result = await response.json();
```

```
        const result = await response.json();
        if (result.status === 'accepted') {
            alert('Override accepted!');
        }
    };

    if (!pendingDecision) return <div>No pending decisions</div>;

    return (
        <div className="override-panel">
            <h3>⚠️ Upcoming Phase Decision</h3>
            <p>Junction: {pendingDecision.junction_id}</p>
            <p>Phase: {pendingDecision.phase_name}</p>
            <p>RL Decision: {pendingDecision.duration}s</p>

            <div className="countdown">
                Override Window: {timeRemaining}s
            </div>

            <input
                type="range"
                min={pendingDecision.duration - 10}
                max={pendingDecision.duration + 10}
                value={newDuration}
                onChange={(e) => setNewDuration(e.target.value)}
            />
            <p>Adjusted Duration: {newDuration}s</p>

            <button onClick={handleOverride}>
                Apply Override
            </button>
        </div>
    );
}
```

5.2.4 Data Model

```python
# models/override_log.py
from dataclasses import dataclass
from datetime import datetime

@dataclass
class OverrideLog:
    timestamp: datetime
    junction_id: str
    phase: int
    rl_decision: int  # seconds
    admin_override: int  # seconds
    operator_id: str
    reason: str  # Optional comment from operator

    def to_dict(self):
        return {
            "timestamp": self.timestamp.isoformat(),
            "junction_id": self.junction_id,
            "phase": phase,
            "rl_decision": self.rl_decision,
            "admin_override": self.admin_override,
            "delta": self.admin_override - self.rl_decision,
            "operator_id": self.operator_id,
            "reason": self.reason
        }
```

5.3 Feature 2: Manual Green-Wave Corridor

5.3.1 User Story

As a traffic control operator
I want to manually create green-wave corridors
So that I can facilitate presidential convoys, emergency response, or special events

Acceptance Criteria:

- Admin can select multiple junctions on city map
- Admin can specify green-wave direction (N→S, E→W, etc.)
- System overrides RL for selected junctions
- Green-wave mode can be disabled to return to normal RL control

5.3.2 Workflow

```
Step 1: Operator selects corridor on map
Admin Panel:
  → Clicks J1, J2, J3 on map
  → Selects direction: "Eastbound (E→W)"
  → Clicks "Activate Green Wave"

Step 2: System computes coordination
Backend:
  → Calculate offsets for progressive timing
  → Example: J1 goes green at t=0s, J2 at t=15s, J3 at t=30s
  → Override RL decisions for EW-phases at all 3 junctions

Step 3: Execution
SUMO Environment:
  → J1: Force Phase 0 (EW-Through) + Phase 1 (EW-Left)
  → Wait 15s
  → J2: Force Phase 0 + Phase 1
  → Wait 15s
  → J3: Force Phase 0 + Phase 1

Step 4: Deactivation
Admin Panel:
  → Clicks "End Green Wave"
  → System returns control to RL agent
```

5.3.3 Green-Wave Algorithm

Progressive Timing Calculation:

python

```python
class GreenWaveController:
    def __init__(self, speed_limit_mps=15):  # 15 m/s ≈ 54 km/h
        self.speed_limit = speed_limit_mps

    def calculate_offsets(self, junctions, direction):
        """
        Calculate time offsets for green wave

        Args:
            junctions: List of junction objects with coordinates
            direction: 'N', 'S', 'E', 'W'

        Returns:
            Dict[junction_id, offset_seconds]
        """
        # Sort junctions by position in direction
        if direction in ['E', 'W']:
            sorted_junctions = sorted(junctions, key=lambda j: j.x)
        else:
            sorted_junctions = sorted(junctions, key=lambda j: j.y)

        if direction in ['W', 'S']:  # Reverse for westbound/southbound
            sorted_junctions.reverse()

        # Calculate offsets based on distance and speed
        offsets = {}
        prev_junction = None
        cumulative_offset = 0

        for junction in sorted_junctions:
            if prev_junction is None:
                offsets[junction.id] = 0  # First junction starts immediately
            else:
                distance = self._distance(prev_junction, junction)
                travel_time = distance / self.speed_limit
                cumulative_offset += travel_time
                offsets[junction.id] = int(cumulative_offset)

            prev_junction = junction

        return offsets

    def _distance(self, j1, j2):
        """Calculate Euclidean distance between junctions"""
        return ((j2.x - j1.x)**2 + (j2.y - j1.y)**2)**0.5
```

```python
    return ((j2.x - j1.x)**2 + (j2.y - j1.y)**2)**0.5

    async def activate_green_wave(self, junction_ids, direction):
        """
        Activate green wave on specified corridor
        """
        junctions = [get_junction(jid) for jid in junction_ids]
        offsets = self.calculate_offsets(junctions, direction)

        # Determine which phases to activate
        phase_map = {
            'E': [0, 1],  # EW-Through + EW-Left
            'W': [0, 1],
            'N': [2, 3],  # NS-Through + NS-Left
            'S': [2, 3]
        }
        phases = phase_map[direction]

        # Schedule green-wave executions
        for junction_id, offset in offsets.items():
            await asyncio.sleep(offset)
            await self._force_phases(junction_id, phases, duration=45)

    async def _force_phases(self, junction_id, phases, duration):
        """Override RL and execute specified phases"""
        for phase in phases:
            await execute_phase_override(junction_id, phase, duration)
```

5.3.4 UI Design

City Map Visualizer:

javascript

```javascript
// GreenWaveBuilder.jsx
import React, { useState } from 'react';
import { MapContainer, TileLayer, Marker, Polyline } from 'react-leaflet';

function GreenWaveBuilder({ junctions }) {
    const [selectedJunctions, setSelectedJunctions] = useState([]);
    const [direction, setDirection] = useState('E');
    const [activeWave, setActiveWave] = useState(null);

    const handleJunctionClick = (junctionId) => {
        setSelectedJunctions(prev =>
            prev.includes(junctionId)
                ? prev.filter(id => id !== junctionId)
                : [...prev, junctionId]
        );
    };

    const activateGreenWave = async () => {
        const response = await fetch('/greenwave/activate', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
                junction_ids: selectedJunctions,
                direction: direction
            })
        });

        const result = await response.json();
        setActiveWave(result.wave_id);
    };

    const deactivateGreenWave = async () => {
        await fetch(`/greenwave/${activeWave}/deactivate`, {
            method: 'POST'
        });
        setActiveWave(null);
        setSelectedJunctions([]);
    };

    return (
        <div className="greenwave-builder">
            <MapContainer center={[28.6139, 77.2090]} zoom={13}>
                <TileLayer url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"

                {junctions.map(junction => (
```

```
        {junctions.map(junction => (
            <Marker
                key={junction.id}
                position={[junction.lat, junction.lng]}
                eventHandlers={{
                    click: () => handleJunctionClick(junction.id)
                }}
                icon={selectedJunctions.includes(junction.id)
                    ? selectedIcon
                    : defaultIcon
                }
            />
        ))}

        {selectedJunctions.length > 1 && (
            <Polyline
                positions={selectedJunctions.map(id =>
                    junctions.find(j => j.id === id)
                )}
                color="green"
                weight={5}
            />
        )}
    </MapContainer>

    <div className="controls">
        <h3>Green Wave Control</h3>
        <p>Selected Junctions: {selectedJunctions.length}</p>

        <select value={direction} onChange={e => setDirection(e.target.value
            <option value="E">Eastbound</option>
            <option value="W">Westbound</option>
            <option value="N">Northbound</option>
            <option value="S">Southbound</option>
        </select>

        {!activeWave ? (
            <button
                onClick={activateGreenWave}
                disabled={selectedJunctions.length < 2}
            >
                Activate Green Wave
            </button>
        ) : (
            <button onClick={deactivateGreenWave}>
                End Green Wave
            </button>
```

```
                )}
            </div>
        </div>
    );
}
```

5.4 Feature 3: Live Traffic Dashboard

5.4.1 Real-Time Metrics Display

Metrics to Show:

javascript

```javascript
// LiveDashboard.jsx
function LiveDashboard() {
    const [metrics, setMetrics] = useState({});

    useEffect(() => {
        const ws = new WebSocket('ws://localhost:8000/ws/metrics');

        ws.onmessage = (event) => {
            const data = JSON.parse(event.data);
            setMetrics(data);
        };

        return () => ws.close();
    }, []);

    return (
        <div className="dashboard">
            <h2>Live Traffic Metrics</h2>

            <div className="metrics-grid">
                {Object.entries(metrics).map(([junctionId, data]) => (
                    <JunctionCard
                        key={junctionId}
                        id={junctionId}
                        queues={data.queues}
                        waitTimes={data.wait_times}
                        currentPhase={data.current_phase}
                        nextDecision={data.next_decision}
                        emergency={data.emergency_flag}
                    />
                ))}
            </div>
        </div>
    );
}

function JunctionCard({ id, queues, waitTimes, currentPhase, nextDecision, emergency
    const phaseNames = ['EW-Through', 'EW-Left', 'NS-Through', 'NS-Left'];

    return (
        <div className={`junction-card ${emergency ? 'emergency' : ''}`}>
            <h3>Junction {id}</h3>

            {emergency && (
```

```
                <div className="emergency-alert">
                    🚨 Emergency Vehicle Detected
                </div>
            )}

            <div className="current-state">
                <p><strong>Current Phase:</strong> {phaseNames[currentPhase]}</p>
                <p><strong>Time Remaining:</strong> {nextDecision.time_remaining}s</
            </div>

            <div className="queues">
                <h4>Queue Lengths</h4>
                <div className="queue-bars">
                    <div className="queue-bar">N: {queues.N} vehicles</div>
                    <div className="queue-bar">S: {queues.S} vehicles</div>
                    <div className="queue-bar">E: {queues.E} vehicles</div>
                    <div className="queue-bar">W: {queues.W} vehicles</div>
                </div>
            </div>

            <div className="wait-times">
                <h4>Average Wait Times</h4>
                <p>N: {waitTimes.N}s | S: {waitTimes.S}s</p>
                <p>E: {waitTimes.E}s | W: {waitTimes.W}s</p>
            </div>

            <div className="next-decision">
                <h4>Upcoming Decision</h4>
                <p>{phaseNames[nextDecision.phase]}: {nextDecision.duration}s</p>
                <button>Override</button>
            </div>
        </div>
    );
}
```

5.5 Technical Stack

Backend:

- **FastAPI** — WebSocket server + REST API
- **asyncio** — Asynchronous execution and coordination
- **SQLite** — Override log storage
- **Redis** — Real-time state management

Frontend:

- **React** — UI framework
- **Leaflet/MapboxGL** — City map visualization
- **Chart.js** — Live metrics charts
- **WebSocket** — Real-time updates

Deployment:

- **Docker** — Containerized deployment
- **Nginx** — Reverse proxy
- **SSL/TLS** — Secure connections

5.6 Security & Access Control

Authentication:

```python
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

async def get_current_operator(token: str = Depends(oauth2_scheme)):
    operator = verify_token(token)
    if not operator:
        raise HTTPException(status_code=401, detail="Invalid credentials")
    return operator

@app.post("/override/{junction_id}")
async def submit_override(
    junction_id: str,
    new_duration: int,
    operator: Operator = Depends(get_current_operator)
):
    # Only authorized operators can override
    if not operator.has_permission("override"):
        raise HTTPException(status_code=403, detail="Insufficient permissions")

    result = override_manager.submit_override(junction_id, new_duration, operator.id
    return result
```

Permission Levels:

Role	Override	Green-Wave	View Dashboard
Operator	✅	❌	✅
Supervisor	✅	✅	✅
Administrator	✅	✅	✅
Read-Only	❌	❌	✅

5.7 Admin Panel Development Checklist

Week 4 Tasks:

- [ ] **Day 1: Backend Setup**
- [ ] Set up FastAPI project
- [ ] Implement WebSocket server
- [ ] Create OverrideManager class
- [ ] Add authentication system
- [ ] **Day 2: Override Feature**
- [ ] Implement decision broadcast
- [ ] Add override submission endpoint
- [ ] Create override logging
- [ ] Write unit tests
- [ ] **Day 3: Green-Wave Feature**
- [ ] Implement GreenWaveController
- [ ] Add offset calculation algorithm
- [ ] Create activation/deactivation endpoints
- [ ] Test with 3-junction corridor
- [ ] **Day 4-5: Frontend Development**
- [ ] Set up React project
- [ ] Build city map visualizer
- [ ] Create live dashboard components
- [ ] Implement override UI
- [ ] Build green-wave builder
- [ ] **Day 6: Integration Testing**
- [ ] End-to-end testing
- [ ] Load testing (multiple admins)
- [ ] Failover scenarios
- [ ] Performance optimization
- [ ] **Day 7: Demo Preparation**
- [ ] Polish UI/UX
- [ ] Create demo scenario
- [ ] Prepare presentation
- [ ] Documentation

---

6. Technical Architecture

6.1 System Components Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                          Admin Panel (Web UI)                        │
│  ┌──────────────┐   ┌──────────────┐   ┌──────────────┐             │
│  │  Map View    │   │  Dashboard   │   │ Green Wave   │             │
│  └──────┬───────┘   └──────┬───────┘   └──────┬───────┘             │
│         │                  │                  │                     │
│         └──────────────────┴──────────────────┘                     │
│                            │ WebSocket                              │
└────────────────────────────┼────────────────────────────────────────┘
                             │
┌────────────────────────────┼────────────────────────────────────────┐
│                  Control Server (FastAPI)                           │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  WebSocket Handler                                           │   │
│  │  - Broadcast RL decisions                                    │   │
│  │  - Receive admin overrides                                   │   │
│  │  - Stream live metrics                                       │   │
│  └──────────────────────────────────────────────────────────────┘   │
│  ┌──────────────┐   ┌──────────────┐   ┌──────────────┐             │
│  │  Override    │   │  Green Wave  │   │  Metrics     │             │
│  │  Manager     │   │  Controller  │   │  Aggregator  │             │
│  └──────┬───────┘   └──────┬───────┘   └──────┬───────┘             │
│         │                  │                  │                     │
│         └──────────────────┴──────────────────┘                     │
│                            │ Python API                             │
└────────────────────────────┼────────────────────────────────────────┘
                             │
┌────────────────────────────┼────────────────────────────────────────┐
│                   RL Agent (PPO + ILPS)                             │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  Decision Loop (Every 20-35s)                                │   │
│  │  1. Observe current junction state [18 dims]                 │   │
│  │  2. Predict action (duration index 0-7)                      │   │
│  │  3. Send decision to Control Server                          │   │
│  │  4. Wait for override (5s window)                            │   │
│  │  5. Execute phase with final duration                        │   │
│  └──────────────────────────────────────────────────────────────┘   │
│  ┌──────────────┐   ┌──────────────┐   ┌──────────────┐             │
│  │  Junction 1  │   │  Junction 2  │   │  Junction 3  │             │
│  │  Brain       │   │  Brain       │   │  Brain       │             │
│  │  (Shared)    │   │  (Shared)    │   │  (Shared)    │             │
│  └──────┬───────┘   └──────┬───────┘   └──────┬───────┘             │
│         │                  │                  │                     │
│         └──────────────────┴──────────────────┘                     │
│                            │ TraCI                                  │
└────────────────────────────┼────────────────────────────────────────┘
```

```
                              │
                              │
┌─────────────────────────────┼──────────────────────────────────────┐
│                   SUMO Traffic Simulator                            │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐             │
│  │  Junction 1  │──│  Junction 2  │──│  Junction 3  │             │
│  │  (J1)        │  │  (J2)        │  │  (J3)        │             │
│  └──────────────┘  └──────────────┘  └──────────────┘             │
│                                                                    │
│         Traffic Flow Simulation (Vehicles, Queues, Waits)          │
└────────────────────────────────────────────────────────────────────┘
```

6.2 Data Flow

Normal Operation (RL Control):

```
1. SUMO updates vehicle positions
2. RL agent observes state
3. Agent predicts action (duration)
4. Decision sent to Control Server
5. Control Server broadcasts to Admin Panel
6. Wait 5s (override window)
7. If no override: Execute RL decision
   If override: Execute admin decision
8. Phase executes in SUMO
9. Repeat
```

Override Mode:

```
1-4. [Same as normal]
5. Admin Panel displays decision
6. Operator adjusts slider within 5s
7. Admin Panel sends override to Control Server
8. Override Manager updates decision
9. Phase executes with overridden duration
10. Log override event
11. Repeat
```

Green-Wave Mode:

```
1. Operator selects junctions on map
2. Operator chooses direction (E/W/N/S)
3. Admin Panel sends green-wave request
4. Green-Wave Controller calculates offsets
5. Control Server disables RL for selected junctions
6. Execute progressive green phases with offsets
7. Continue until operator deactivates
8. Re-enable RL control
```

6.3 State Management

Shared State (Redis):

```python
{
    "junctions": {
        "J1": {
            "current_phase": 2,
            "time_remaining": 18,
            "queues": {"N": 5, "S": 3, "E": 12, "W": 8},
            "wait_times": {"N": 15.2, "S": 10.5, "E": 25.3, "W": 18.7},
            "emergency_flag": false,
            "next_decision": {
                "phase": 3,
                "duration": 35,
                "timestamp": "2026-03-15T10:23:45"
            },
            "override_active": false
        },
        "J2": { ... },
        "J3": { ... }
    },
    "green_wave": {
        "active": false,
        "junction_ids": [],
        "direction": null,
        "started_at": null
    },
    "system_status": "NORMAL"  # NORMAL | OVERRIDE | GREEN_WAVE
}
```

---

7. Implementation Timeline

7.1 Overall Schedule

Total Duration: 4 weeks
Start Date: March 18, 2026
Demo Date: April 15, 2026 (Smart India Hackathon)

```
Week 1 (Mar 18-24): Phase 1 Part A - ILPS Environment
Week 2 (Mar 25-31): Phase 1 Part B - ILPS Training & Validation
Week 3 (Apr 1-7):   Phase 1 Part C - Optimization & Evaluation
Week 4 (Apr 8-14):  Phase 2 - Admin Panel Development
```

7.2 Detailed Week-by-Week Breakdown

Week 1: ILPS Environment Refactoring

Monday (Day 1):

- [ ] Morning: Design review of ILPS architecture
- [ ] Afternoon: Create `PerPhaseCorridorEnv` skeleton
- [ ] Evening: Implement basic observation space (18 dims)

Tuesday (Day 2):

- [ ] Morning: Implement neighbor phase tracking
- [ ] Afternoon: Update reward calculation for per-phase
- [ ] Evening: Write unit tests for environment

Wednesday (Day 3):

- [ ] Morning: Implement per-phase decision loop
- [ ] Afternoon: Add MIN_GREEN=20s constraint
- [ ] Evening: Test single junction scenario

Thursday (Day 4):

- [ ] Morning: Extend to 3-junction corridor
- [ ] Afternoon: Debug coordination issues
- [ ] Evening: Performance profiling

Friday (Day 5):

- [ ] Morning: Update training script for ILPS
- [ ] Afternoon: Implement curriculum learning stages
- [ ] Evening: Configure TensorBoard logging

Weekend (Day 6-7):

- [ ] Saturday: Code review and refactoring
- [ ] Sunday: Documentation and prep for Week 2

Week 2: ILPS Training & Validation

Monday (Day 8):

- [ ] Morning: Start Stage 1 training (single junction, 66K steps)
- [ ] Afternoon: Monitor training, adjust hyperparameters
- [ ] Evening: Stage 1 completion check

Tuesday (Day 9):

- [ ] Morning: Start Stage 2 training (3 junctions, 66K steps)
- [ ] Afternoon: Monitor green-wave emergence
- [ ] Evening: Validate coordination behavior

Wednesday (Day 10):

- [ ] Morning: Start Stage 3 training (heavy traffic, 68K steps)
- [ ] Afternoon: Monitor safety metrics
- [ ] Evening: Training completion (200K total)

Thursday (Day 11):

- [ ] Morning: Validate on 3-junction corridor
- [ ] Afternoon: Test on 9-junction grid (zero retraining!)
- [ ] Evening: Performance comparison vs centralized

Friday (Day 12):

- [ ] Morning: Hyperparameter tuning if needed
- [ ] Afternoon: Re-train if performance below target
- [ ] Evening: Checkpoint best model

Weekend (Day 13-14):

- [ ] Saturday: Extended testing and stress scenarios
- [ ] Sunday: Prepare Week 3 evaluation plan

Week 3: Optimization & Evaluation

Monday (Day 15):

- [ ] Morning: Run full evaluation (20 eps × 3 scenarios)
- [ ] Afternoon: Analyze results vs baselines
- [ ] Evening: Document performance improvements

Tuesday (Day 16):

- [ ] Morning: Stress test with 25-junction grid
- [ ] Afternoon: Memory optimization for large grids
- [ ] Evening: Scalability validation

Wednesday (Day 17):

- [ ] Morning: Emergency vehicle scenario testing
- [ ] Afternoon: Random incident testing
- [ ] Evening: Edge case handling

Thursday (Day 18):

- [ ] Morning: Code cleanup and refactoring
- [ ] Afternoon: Write ILPS technical documentation
- [ ] Evening: Update README and guides

Friday (Day 19):

- [ ] Morning: Prepare demo for stakeholders
- [ ] Afternoon: Internal presentation
- [ ] Evening: Gather feedback

Weekend (Day 20-21):

- [ ] Saturday: Address feedback
- [ ] Sunday: Phase 1 completion review

Week 4: Admin Panel Development

Monday (Day 22):

- [ ] Morning: Set up FastAPI project structure
- [ ] Afternoon: Implement WebSocket server
- [ ] Evening: Create OverrideManager class

Tuesday (Day 23):

- [ ] Morning: Implement decision broadcast logic
- [ ] Afternoon: Add override submission endpoint
- [ ] Evening: Create override logging system

Wednesday (Day 24):

- [ ] Morning: Implement GreenWaveController
- [ ] Afternoon: Add offset calculation algorithm
- [ ] Evening: Test green-wave on 3-junction corridor

Thursday (Day 25):

- [ ] Morning: Set up React frontend project
- [ ] Afternoon: Build city map visualizer (Leaflet)
- [ ] Evening: Implement junction selection UI

Friday (Day 26):

- [ ] Morning: Create live dashboard components
- [ ] Afternoon: Implement override slider UI
- [ ] Evening: Build green-wave builder interface

Weekend (Day 27-28):

- [ ] Saturday: End-to-end integration testing
- [ ] Sunday: UI/UX polish and bug fixes

Monday (Day 29) - DEMO DAY:

- [ ] Morning: Final testing and rehearsal
- [ ] Afternoon: Smart India Hackathon presentation
- [ ] Evening: Celebration! 🎉

7.3 Risk Mitigation Buffers

Built-in Slack:

- Week 2 Weekend: 2 days for training issues
- Week 3 Weekend: 2 days for performance tuning
- Week 4 Thursday-Friday: 2 days for admin panel bugs

Fallback Plans:

If ILPS training fails: → Fall back to centralized 3-junction model (already proven)

If admin panel not ready: → Demo with CLI override commands (less flashy but functional)

If green-wave algorithm buggy: → Demo manual per-junction override instead

---

8. Success Metrics

8.1 Phase 1 Success Criteria

Functional Requirements:

Requirement	Target	Measurement
ILPS trains successfully	200K steps without crash	Training logs
Per-phase decisions work	100% of steps	Environment tests
Scalability validated	3→9→25 junctions	Deployment tests
MIN_GREEN=20s enforced	100% of phases	Phase duration logs

Performance Requirements:

Metric	Target	Baseline
Wait Time vs Max-Pressure	⩾ -50%	-58% (current)
Queue Length vs Max-Pressure	⩾ -45%	-53% (current)
Response Time	⩽ 35s	45-90s (current)
Starvation Events	0	0 (current)

Quality Requirements:

- [ ] Code coverage ⩾ 80%
- [ ] Documentation complete
- [ ] No critical bugs
- [ ] Performance profiling done

8.2 Phase 2 Success Criteria

Override Feature:

Requirement	Target
Override window	5 seconds
Override success rate	⩾ 95%
Override latency	< 200ms
Audit log completeness	100%

Green-Wave Feature:

Requirement	Target
Corridor creation time	< 10s
Offset calculation accuracy	⩽ ±2s error
Activation latency	< 1s
Deactivation recovery	< 5s

Admin Panel Performance:

Metric	Target
Page load time	< 2s
WebSocket latency	< 100ms
Concurrent admins	⩾ 5
Dashboard refresh rate	1 Hz

8.3 Demo Success Criteria

Hackathon Presentation Goals:

- [ ] **Impress judges** with live ILPS scalability demo
- [ ] **Show admin override** working in real-time
- [ ] **Demonstrate green-wave** for convoy scenario
- [ ] **Prove performance** with comparison charts
- [ ] **Answer questions** confidently about architecture

Audience Engagement:

- [ ] Non-technical judges understand the value proposition
- [ ] Technical judges impressed by ILPS architecture
- [ ] Questions about deployment readiness (good sign!)
- [ ] Interest from industry sponsors
- [ ] Top 3 placement in hackathon

---

9. Risk Assessment

9.1 Technical Risks

Risk 1: ILPS Training Convergence Issues

Probability: Medium (30%)
Impact: High

Description: Per-phase decisions may be harder to learn than batch decisions. Agent might struggle to coordinate across junctions without global observation.

Mitigation:

- Start with single junction (simpler)
- Use curriculum learning (proven to work)
- Add neighbor phase features to help coordination
- Fall back to centralized model if needed

Contingency: If ILPS doesn't converge by Week 2 end, continue with centralized 3-junction model for demo.

Risk 2: Admin Override Race Conditions

Probability: Medium (25%)
Impact: Medium

Description: Multiple admins trying to override the same junction simultaneously could cause conflicts or missed overrides.

Mitigation:

- Implement mutex locking per junction
- First-come-first-served override policy
- Clear UI indication when another admin has control
- Audit logging of all override attempts

Contingency: Restrict to single admin for demo if multi-admin bugs persist.

Risk 3: Green-Wave Offset Calculation Errors

Probability: Low (15%)
Impact: Medium

Description: Progressive timing calculation may be inaccurate for real vehicle speeds or complex road geometries.

Mitigation:

- Use conservative speed estimates (54 km/h urban)
- Add manual offset adjustment in UI
- Test with multiple scenarios
- Validate with SUMO vehicle travel times

Contingency: Allow manual offset input if automatic calculation is buggy.

9.2 Schedule Risks

Risk 4: Training Takes Longer Than Expected

Probability: Medium (35%)
Impact: Medium

Description: 200K steps might not be enough for ILPS to learn coordination. May need 300-500K steps.

Mitigation:

- Start training early (Week 1)
- Use 4 parallel environments (faster)
- Monitor convergence closely
- Hyperparameter tuning

Contingency: Use best checkpoint even if not fully converged. Extend training into Week 3 if needed.

Risk 5: Admin Panel UI Takes Too Long

Probability: High (40%)
Impact: Low

Description: Frontend development often takes longer than expected. Polish and UX may suffer.

Mitigation:

- Use pre-built UI components (React-Leaflet, MUI)
- Focus on functionality over beauty
- Simplify features if time-constrained
- Prepare low-fi mockups early

Contingency: Demo with minimal UI (basic map + override buttons). Polish post-hackathon.

9.3 Demo Risks

Risk 6: Live Demo Technical Failure

Probability: Medium (30%)
Impact: High

Description: WebSocket disconnects, SUMO crashes, or RL agent freezes during live demo.

Mitigation:

- Record backup demo video
- Test thoroughly on demo hardware
- Prepare manual failover procedures
- Have offline mode ready

Contingency: Switch to recorded video if live demo fails. Explain issues honestly.

Risk 7: Judges Don't Understand Technical Depth

Probability: Low (20%)
Impact: Medium

Description: Non-technical judges may not appreciate ILPS architecture complexity or admin panel value.

Mitigation:

- Prepare 2-minute non-technical pitch
- Use visual demos (map, charts)
- Emphasize 58% improvement clearly
- Show convoy green-wave (impressive visual)

Contingency: Have technical Q&A prepared for judges who do understand.

---

10. Appendix: Technical Specifications

10.1 ILPS Environment Specification

Class Definition:

python

```python
class PerPhaseCorridorEnv(gym.Env):
    """
    Per-phase multi-junction environment with ILPS support.

    Each step controls ONE phase at ONE junction.
    Observation includes local junction state + neighbor info.
    """

    metadata = {'render.modes': ['human']}

    def __init__(
        self,
        n_junctions: int = 3,
        sumo_cfg: str = "configs/sumo/corridor.sumocfg",
        use_gui: bool = False,
        curriculum_stage: int = 0
    ):
        super().__init__()

        # ILPS observation: local (18) + junction_id (1)
        self.observation_space = spaces.Box(
            low=0, high=1,
            shape=(19,),
            dtype=np.float32
        )

        # ILPS action: single duration index
        self.action_space = spaces.Discrete(8)

        self.n_junctions = n_junctions
        self.junction_ids = [f"J{i+1}" for i in range(n_junctions)]

        # Phase execution state
        self.current_junction_idx = 0
        self.current_phase_idx = 0

        # SUMO environment
        self.sumo = SUMOSimulation(sumo_cfg, use_gui)

        # Reward calculator
        self.reward_calc = EnhancedRewardCalculator()

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
```

```
        # Generate new traffic scenario
        route_file = generate_route_file(curriculum_stage)
        self.sumo.start(route_file)

        # Reset phase tracking
        self.current_junction_idx = 0
        self.current_phase_idx = 0

        obs = self._get_observation()
        info = {}

        return obs, info

    def step(self, action):
        # Decode action
        duration_idx = action
        duration = signal.GREEN_DURATIONS[duration_idx]

        # Get current junction
        junction_id = self.junction_ids[self.current_junction_idx]

        # Execute phase
        self._execute_phase(junction_id, self.current_phase_idx, duration)

        # Calculate reward
        reward = self._calculate_reward(junction_id)

        # Move to next junction for same phase
        self.current_junction_idx += 1

        # If completed all junctions, move to next phase
        if self.current_junction_idx >= self.n_junctions:
            self.current_junction_idx = 0
            self.current_phase_idx = (self.current_phase_idx + 1) % 4

        # Get next observation
        obs = self._get_observation()

        # Check termination
        terminated = self.sumo.get_current_time() >= simulation.EPISODE_LENGTH
        truncated = False

        info = {
            "junction_id": junction_id,
            "phase": self.current_phase_idx,
            "duration": duration
```

```python
        }

        return obs, reward, terminated, truncated, info

    def _get_observation(self):
        junction_id = self.junction_ids[self.current_junction_idx]

        # Local state (14 dims)
        state = self._get_junction_state(junction_id)
        queues = state['queues']  # [N, S, E, W]
        deltas = state['deltas']
        waits = state['waits']
        emergency = state['emergency_flag']
        phase = self.current_phase_idx / 3.0  # Normalize

        # Neighbor states (4 dims)
        upstream_state = self._get_neighbor_state(self.current_junction_idx - 1)
        downstream_state = self._get_neighbor_state(self.current_junction_idx + 1)

        # Combine
        obs = np.array([
            *queues, *deltas, *waits,
            emergency, phase,
            upstream_state['phase'], upstream_state['time_since_switch'],
            downstream_state['phase'], downstream_state['time_since_switch'],
            self.current_junction_idx / (self.n_junctions - 1)  # Junction ID
        ], dtype=np.float32)

        return obs

    def _get_neighbor_state(self, neighbor_idx):
        if neighbor_idx < 0 or neighbor_idx >= self.n_junctions:
            return {'phase': -1, 'time_since_switch': -1}

        neighbor_id = self.junction_ids[neighbor_idx]
        # ... fetch from SUMO
        return {'phase': phase_norm, 'time_since_switch': time_norm}
```

10.2 Admin Panel API Specification

WebSocket Messages:

```json
// Server → Client: Decision Broadcast
{
    "type": "decision",
    "junction_id": "J2",
    "phase": 2,
    "phase_name": "NS-Through",
    "duration": 35,
    "timestamp": "2026-03-15T10:23:45.123Z",
    "override_deadline": "2026-03-15T10:23:50.123Z"
}

// Client → Server: Override Submission
{
    "type": "override",
    "junction_id": "J2",
    "new_duration": 45,
    "operator_id": "admin_001",
    "reason": "Heavy NS queue observed on camera"
}

// Server → Client: Override Confirmation
{
    "type": "override_result",
    "status": "accepted",
    "junction_id": "J2",
    "old_duration": 35,
    "new_duration": 45,
    "timestamp": "2026-03-15T10:23:48.456Z"
}

// Server → Client: Metrics Update (1 Hz)
{
    "type": "metrics",
    "junctions": {
        "J1": {
            "queues": {"N": 5, "S": 3, "E": 12, "W": 8},
            "wait_times": {"N": 15.2, "S": 10.5, "E": 25.3, "W": 18.7},
            "current_phase": 2,
            "time_remaining": 18,
            "emergency_flag": false
        },
        ...
    },
    "timestamp": "2026-03-15T10:23:45.789Z"
}
```

```
}
```

REST API Endpoints:

```
POST /greenwave/activate
Body: {
    "junction_ids": ["J1", "J2", "J3"],
    "direction": "E",  // E, W, N, S
    "operator_id": "admin_001"
}
Response: {
    "wave_id": "wave_12345",
    "offsets": {"J1": 0, "J2": 15, "J3": 30},
    "status": "activated"
}

POST /greenwave/{wave_id}/deactivate
Response: {
    "status": "deactivated",
    "ended_at": "2026-03-15T10:25:00.000Z"
}

GET /override/history
Query: ?from=timestamp&to=timestamp&junction_id=J1
Response: [
    {
        "timestamp": "2026-03-15T10:23:48.456Z",
        "junction_id": "J2",
        "phase": 2,
        "rl_decision": 35,
        "admin_override": 45,
        "delta": 10,
        "operator_id": "admin_001",
        "reason": "Heavy NS queue"
    },
    ...
]

GET /metrics/current
Response: { ... } // Same as WebSocket metrics message
```

10.3 Configuration Changes

New Parameters:

```python
# In configs/parameters.py

class SignalConfig:
    MIN_GREEN = 20  # Increased from 10
    GREEN_DURATIONS = [20, 25, 30, 35, 40, 45, 50, 55]  # Updated

class ILPSConfig:
    ENABLE_ILPS = True
    NEIGHBOR_OBSERVATION = True  # Include upstream/downstream phase
    PER_PHASE_DECISION = True  # One phase at a time

class AdminPanelConfig:
    OVERRIDE_WINDOW_SECONDS = 5
    MAX_OVERRIDE_DELTA = 10  # ±10 seconds
    WEBSOCKET_PORT = 8000
    METRICS_UPDATE_RATE = 1  # Hz
    ENABLE_GREEN_WAVE = True
    DEFAULT_SPEED_MPS = 15  # For offset calculation
```

End of PRD

Next Steps:

1. Review this PRD with team
2. Approve architecture decisions
3. Begin Week 1 implementation
4. Set up GitHub project board for tracking

Questions or Concerns: Contact: [Your Email]
Last Updated: March 11, 2026
Version: 1.0